



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

Titulo TFG



Presentado por Lydia Blanco Ruiz
en Universidad de Burgos — 8 de febrero
de 2026

Tutor: Nombre Tutor
y Nombre Co-tutor



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. Nombre Tutor y Nombre Co-tutor, profesores del departamento de Ingeniería Informática, área de nombre área.

Expone:

Que el alumno D. Lydia Blanco Ruiz, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado Título TFG.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 8 de febrero de 2026

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

D. Nombre Tutor

D. Nombre Co-tutor

Resumen

En este primer apartado se hace una **breve** presentación del tema que se aborda en el proyecto.

Descriptores

Palabras separadas por comas que identifiquen el contenido del proyecto Ej: servidor web, buscador de vuelos, android ...

Abstract

A **brief** presentation of the topic addressed in the project.

Keywords

keywords separated by commas.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
1. Introducción	1
2. Objetivos del proyecto	3
2.1. Objetivos específicos	3
2.2. Objetivos personales	4
3. Conceptos teóricos	5
3.1. <i>Web Scraping</i>	5
3.2. <i>LLM</i>	5
3.3. <i>RAG</i>	7
3.4. Conexión del <i>LLM</i> con el <i>RAG</i>	8
3.5. Diseño del <i>RAG</i>	9
3.6. Evaluación de la calidad del <i>RAG</i>	13
4. Técnicas y herramientas	17
4.1. Metodologías	17
4.2. Repositorio	17
4.3. Gestión de tareas	18
4.4. $\text{\LaTeX}$	19
4.5. <i>Web Scraping</i>	19
4.6. Gestión de dependencias y entornos <i>Python</i>	19

4.7. Base de datos vectorial	21
4.8. Postgres	23
4.9. Docker	23
5. Aspectos relevantes del desarrollo del proyecto	25
6. Trabajos relacionados	27
6.1. TFG relacionados	27
7. Conclusiones y Líneas de trabajo futuras	29
Bibliografía	31

Índice de figuras

3.1. Diagrama del funcionamiento del <i>RAG</i>	7
3.2. Funcionamiento de un modelo <i>LLM</i> con <i>RAG</i> y otro sin él [1]. .	9
3.3. Triada de evaluación TruEra.	14
4.1. Metodología <i>Scrum</i>	18

Índice de tablas

4.1. Ventajas y desventajas de <i>Selenium</i> y <i>Playwright</i>	20
4.2. Ventajas y desventajas de <i>Qdrant</i> y <i>pgvector</i>	23

1. Introducción

Descripción del contenido del trabajo y del estructura de la memoria y del resto de materiales entregados.

2. Objetivos del proyecto

El objetivo principal del proyecto es diseñar e implementar un sistema de *Retrieval-Augmented Generation* (RAG) que permita enriquecer las respuestas de un modelo de lenguaje mediante la recuperación y utilización de información contextual proveniente de una colección documental.

2.1. Objetivos específicos

Los objetivos identificados en el presente trabajo fin de grado son los siguientes:

- Recopilar la colección documental.
 - Automatizar la obtención de los documentos (licitaciones del estado) a través de *Web Scraping*.
- Preprocesar la colección documental.
 - Segmentar los documentos en fragmentos (*chunks*).
 - Normalizar y limpiar el texto.
 - Generar *embeddings* para representar semánticamente los fragmentos.
- Implementar y gestionar un índice vectorial.
 - Almacenar los *embeddings* en una base de datos vectorial.
 - Optimizar consultas de similitud para recuperar contexto relevante.
- Desarrollar la capa de interacción con el usuario
 - Transformar la consulta en su representación vectorial.

- Recuperar los fragmentos más relevantes del índice.
- Integrar los fragmentos en el *prompt* para enriquecer la respuesta de los *Large Language Model* (LLM).
- Desarrollar interfaz amigable para el usuario.
 - Desarrollar una aplicación con *Flask* para que el usuario pueda interactuar con el modelo.
 - Levantar la aplicación *Flask* con *Docker* para no depender del *requirements.txt*

2.2. Objetivos personales

- Adquirir conocimientos sobre los sistemas RAG, comprendiendo sus fundamentos, componentes y aplicaciones en el ámbito de la inteligencia artificial.
- Profundizar en el estudio de los modelos de lenguaje de gran tamaño (LLM).
- Desarrollar habilidades prácticas en la implementación de sistemas basados en RAG y LLM.
- Aprender a aplicar los sistemas RAG y los LLM en la resolución de problemas reales.
- Aplicar los conocimientos adquiridos durante la carrera.

3. Conceptos teóricos

En este apartado se van a definir los conceptos más relevantes a tener en cuenta sobre el proyecto, por lo cual se considera importante definir qué es el *Web Scraping*, que es un *RAG* y cuáles son sus aportaciones más significativas a los *LLM* que ya existen. Además se considera importante contextualizar su origen, y para ello se va a comenzar explicando que son los *LLM* y sus principales limitaciones.

3.1. *Web Scraping*

Web Scraping es el proceso automático de extraer información expuesta en páginas web y transformarla en datos estructurados. Se realiza mediante programas que descargan y analizan el código HTML o emulan la navegación de un usuario con un navegador controlado por código. Se apoya en técnicas de rastreo (*crawling*), *parseo* y normalización para preparar el dato para su análisis o integración en sistemas [8].

Existe una convención estándar, llamada *Robots Exclusion Protocol* (`robots.txt`), mediante la cual un servidor web indica qué *URLs* se deben evitar rastrear. En este archivo se especifica qué partes del sistema están permitidas o prohibidas para cada tipo de *bot*. Aunque, el incumplimiento de este protocolo no tiene consecuencias legales es éticamente correcto cumplirlo [5].

3.2. *LLM*

Los *Large Language Model* (*LLM*), que se traduce como los grandes modelos del lenguaje, son modelos de aprendizaje automático que han

sido entrenados con grandes conjuntos de datos generados por humanos. Estos datos se usan para encontrar patrones estadísticos y replicar nuestras habilidades lingüísticas. Por lo cual, se puede decir que en esencia son modelos de predicción del siguiente *token* o lo que es lo mismo de la siguiente palabra [4].

Algunos ejemplos de *LLM* son ChatGPT¹, Claude² o Llama³.

Las principales características de los *LLM* son las siguientes:

- Naturaleza de predicción del próximo token: como se ha mencionado anteriormente los *LLM* seleccionan la palabra más probable de una distribución. La elección de esta palabra se basa en los patrones estadísticos que ha aprendido con los datos del entrenamiento.
- Memoria paramétrica: es el conocimiento interno del *LLM*. Consiste en la información con la que ha sido entrenado y se basa únicamente en sus parámetros. Por lo cual es una memoria limitada y estática.

Las principales limitaciones de los *LLM* son las siguientes:

- Desactualizado: los eventos posteriores al entrenamiento del *LLM*, es decir, la fecha de corte del cocimiento, no estarán disponibles para el modelo lo que puede hacer que la respuesta este desactualizada. Además, el entrenamiento de un *LLM* es muy costoso y lleva mucho tiempo, por lo cual no se reentrenan con mucha frecuencia.
- Alucinaciones: a veces los *LLM* dan respuestas incorrectas con gran confianza de manera que parece que la información que te está dando es verdadera. Esto se debe a su naturaleza de predicción del próximo *token*.
- Limitación de conocimiento: los *LLM* se entrenan con datos públicos, lo que limita su conocimiento, ya que no tienen conocimientos de información que no sea pública, como, por ejemplo, documentos internos de una empresa, información de los clientes o de los productos.

Para solventar las limitaciones que se han comentado la solución es darle al *LLM* el contexto que le falta. El contexto del *LLM* es la información no paramétrica que se le proporciona en la consulta o *prompt* para que el modelo genere una respuesta más precisa, actual y fundamentada. Para

¹OpenAI. ChatGPT. Disponible en: <https://chat.openai.com>.

²Anthropic. Claude. Disponible en: <https://claude.ai>.

³Meta AI. Llama. Disponible en: <https://ai.meta.com/llama/>

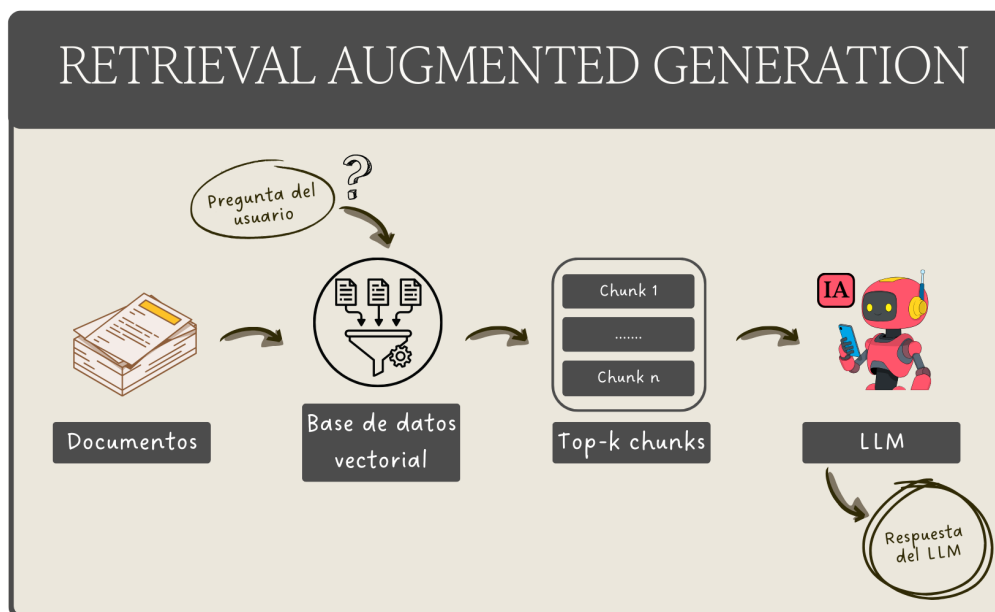


Figura 3.1: Diagrama del funcionamiento del *RAG*.

proporcionarle este contexto se puede expandir la memoria del *LLM* mediante generación aumentada, es decir, incorporando un *RAG*.

3.3. *RAG*

Retrieval Augmented Generation (*RAG*), o en español, generación aumentada por recuperación, es una técnica que consiste en recuperar la información que sea relevante de una fuente externa. Con dicha información aumenta la entrada al *LLM*, lo que permite que el *LLM* genere una respuesta mucho más precisa, contextual y confiable (ver pasos en la imagen 3.1). Para realizar esto, el *RAG* usa tres pasos: recuperar, aumentar y generar.

El *RAG* combina dos tipos de memoria:

- La memoria paramétrica: es la que típicamente tienen los *LLM*, que como se ha visto es limitada y estática.
- La memoria no paramétrica: es información externa al *LLM* que se le puede proporcionar. Es un conocimiento externo y dinámico que se puede extender tanto como se desee y que puede contener documentos propietarios.

Por lo cual conociendo esto, se puede decir que la generación aumentada por recuperación (*RAG*) es una metodología que busca ampliar la memoria paramétrica de un *LLM* incorporando una memoria no paramétrica explícita. A través de un recuperador, se obtiene información relevante de esta memoria externa, la cual se añade al *prompt* y luego se envía al *LLM*. De esta forma, el modelo puede producir respuestas más contextuales, confiables y precisas [4].

Los objetivos de *RAG* son:

- Hacer que los *LLM* respondan con información actualizada.
- Hacer que los *LLM* respondan información precisa, contextual y confiable.
- Hacer que los *LLM* conozcan información propietaria.

Las principales ventajas de *RAG* son:

- Conocimiento del contexto profundo: esto se debe a que la información adicional ayuda al *LLM* a generar respuestas contextualmente apropiadas, ya que, las respuestas se basan en datos específicos. Esto aumenta la confianza del usuario.
- Citar fuentes: como la información se extrae de fuentes conocidas las puede citar en su respuesta. Esto dispara la fiabilidad y permite a los usuarios comprobar la información obtenida.
- Reduce las alucinaciones: el sistema esta anclado a los hechos, por lo que se reducen las respuestas inexactas o falsas.

3.4. Conexión del *LLM* con el *RAG*

El primer paso es el de recopilar los documentos externos y convertirlos a vectores numéricos (*embeddings*) y almacenarlos. Cuando llega una consulta (*prompt*), en lugar de buscar coincidencias por palabras, se busca en ese espacio vectorial los fragmentos que son semánticamente más parecidos, a esto se le conoce como búsqueda por significado. Esos fragmentos se incorporan como contexto al *LLM*, de modo que el *RAG* combina la capacidad de razonamiento del modelo con una recuperación de información altamente precisa. En la imagen 3.2 se puede ver la diferencia de funcionamiento entre un modelo que emplea *RAG* y uno que no lo hace [1].

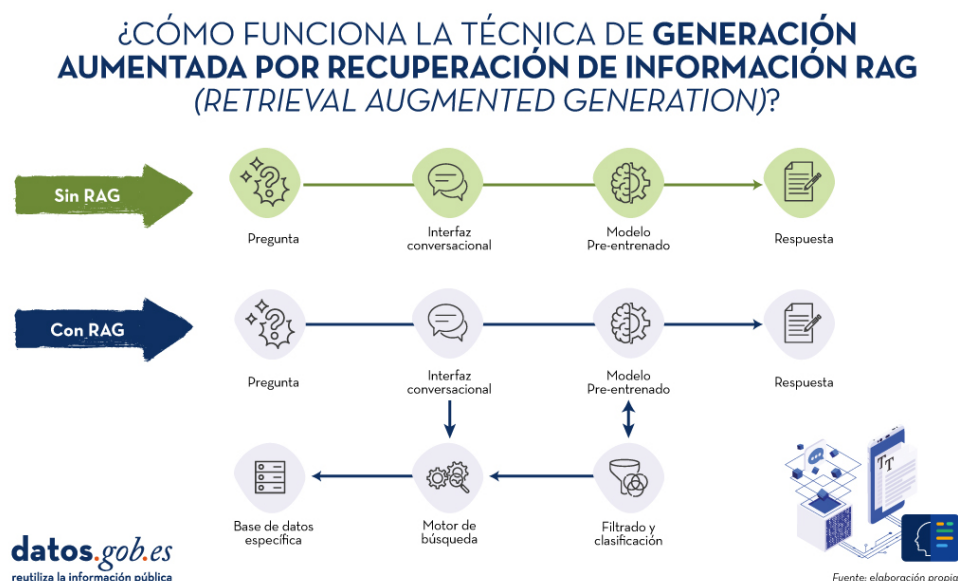


Figura 3.2: Funcionamiento de un modelo *LLM* con *RAG* y otro sin él [1].

3.5. Diseño del *RAG*

Un sistema *RAG* se construye en dos *pipelines* complementarios, el de indexación (*offline*/asíncrono), y el de generación (*online*/ tiempo real). El primero crea y mantiene la memoria no paramétrica o base de conocimiento. El segundo gestiona la interacción con el usuario, realizando la recuperación, el aumento del *prompt* y la generación de la respuesta. Esta separación es estructural en el diseño de *RAG* y permite optimizar la precisión y la latencia del sistema [4].

Pipeline de Indexación (*offline*)

Su objetivo es consolidar y preparar el conocimiento para que el *pipeline* de generación pueda consultarlo eficientemente. Consta de cinco pasos:

1. Conectarse a las fuentes externas.
2. Extraer y *parsear* los documentos.
3. Dividir textos largos en fragmentos más pequeños denominados *chunks*.
4. Convierte los *chunks* a un formato adecuado

5. Almacena en una base de datos vectorial los *embeddings*

Además de crearlo, este *pipeline* mantiene y actualiza el conocimiento base, por lo que debe ejecutarse de forma periódica [4].

Los componentes clave del *pipeline* de indexación son:

- Carga de datos (*data loading*): consiste en conectarse a las fuentes, por ejemplo, sistemas de ficheros, *data lakes*, CMS o APIs, para la extracción del texto en bruto y el parsing de este en formatos heterogéneos, como pueden ser PDF, DOCX, JSON o HTML. Luego se le añaden metadatos y finalmente se hace limpieza quitando código, etiquetas raras y enmascarando datos confidenciales.
- División de texto (*chunking*): segmentación en unidades manejables, los *chunks*, para mejorar la recuperación y el alineamiento con las capacidades de contexto *LLM*. Ayuda a respetar la ventana de contexto, mitiga el problema de pérdida en el medio y facilita la búsqueda eficiente. Para dividir el texto puede usar diversos métodos:
 - Tamaño fijo: divide el texto en longitudes uniformes, como un número de caracteres o *tokens*.
 - Especializados: divide el texto basándose en su estructura como encabezados o párrafos.
 - Semántico: divide el texto basándose en su similitud de significado usando *embeddings*, aunque este método aun es experimental.

La estrategia elegida dependerá del tipo de contenido, la longitud y complejidad de la consulta, el caso de uso y el modelo de *embeddings*.

- Conversión a vectores (*embeddings*): transforma el texto a representaciones numéricas de n dimensiones. Para conocer si los *chunks* se parecen a la consulta, el sistema mide la distancia entre sus vectores, ya que la proximidad refleja similitud semántica. Hay diversas técnicas para medir la distancia entre los vectores, una de las más utilizadas es la similitud coseno que consiste en que si el ángulo entre dos vectores es muy pequeño su similitud es casi uno, pero hay otras como la distancia euclidiana. Hay modelos preentrenados abiertos y propietarios, la elección dependerá de su uso, rendimiento, recuperación y coste.
- Almacenamiento: los *embeddings* se almacenan en bases de datos vectoriales diseñadas para vectores de altas dimensiones. Estas bases de datos vectoriales permiten la búsqueda eficiente sobre *embeddings* y metadatos. Hay diversas opciones:

- Índices vectoriales: son bibliotecas mínimas centradas en indexación y búsqueda de alta dimensión, no gestionan datos ni ofrecen consultas ricas o interfaces de bases de datos. Ejemplos: [FAISS](#), [NMSLIB](#), [ANNOY](#), [ScaNN](#). Son útiles cuando se desea máximo control y rendimiento sin sobrecarga de base de datos.
- Bases de datos vectoriales especializadas: añaden a lo anterior gestión de datos, seguridad, escalado, extensibilidad y metadatos, manteniendo búsqueda/similitud vectorial eficiente. Ejemplos: [Pinecone](#), [ChromaDB](#), [Milvus](#), [Qdrant](#).
- Plataformas de búsqueda: son motores de texto tradicional ([Solr](#), [Elasticsearch](#), [OpenSearch](#), [Lucene](#)) que han incorporado similitud vectorial a sus capacidades.
- Motores SQL/NoSQL con capacidad vectorial: son bases de datos ya existentes que añaden tipos y operadores vectoriales. Ejemplos: [Azure SQL](#), [PostgreSQL/pgvector](#) o en NoSQL [MongoDB](#). Son útiles para consolidar datos, pero son menos especialización que una vector DB dedicada.
- Bases de datos gráficas con funciones vectoriales: grafos como [Neo4j](#) que añaden búsqueda vectorial. Permiten combinar relaciones explícitas (grafos) con similitud semántica (vectores). Ejemplos: *path finding* sujeto a similitud.

Cuando la recuperación se externaliza, es decir, cuando la búsqueda y obtención de información no la realiza el sistema sobre su propia base vectorial, sino que se delegan en un tercero (como por ejemplo una API) o en el documento que aporta el usuario, no es imprescindible construir un *pipeline* de indexación ni mantener una base vectorial. En cambio, en un *RAG* clásico, donde el sistema recupera conocimiento de su repositorio interno, sí se recomienda diseñar el *pipeline* de indexación y mantener una base vectorial propia [4].

Pipeline de Generación (online)

Gestiona la consulta del usuario de extremo a extremo. Este *pipeline* recibe la pregunta del usuario, recupera el contexto relevante de la base vectorial, los *chunks*, aumenta el *prompt* con ese contexto y genera la respuesta con el *LLM*. Se compone de tres fases: *retrieval*, *augmentation* y *generation* [4].

Los componentes clave del *pipeline* de generación son:

- **Retriever:** es el motor de búsqueda sobre la base vectorial, es decir, la base de conocimiento. Devuelve los documentos más relevantes. Es crítico para la precisión del sistema y contribuye a la latencia, su calidad condiciona el rendimiento del *RAG*. El método más frecuente son los *embeddings* gracias a su capacidad semántica.
- **Gestión del *Prompt* (*Augmentation*):** combina la consulta y el contexto recuperado. Además, aplica estrategias de ingeniería de *prompt* para maximizar la calidad de la respuesta. Las estrategias recomendadas son:
 - *Contextual prompting*: consiste en instruir al modelo para que solo responda con el contexto proporcionado.
 - *Controlled generation prompting*: consiste en indicarle al modelo que no conteste si el contexto no permite responder a la pregunta.
 - *Few-shot prompting*: consiste en incluir ejemplos para forzar el formato de la respuesta.
 - *Chain-of-Thought*: consiste en pedir pasos intermedios cuando los razonamientos son complejos.
- **Configuración *LLM* (*Generation*):** es la selección del modelo que va a generar la respuesta final. Pueden ser modelos:
 - Originales o *fine-tuned*: los originales facilitan un arranque rápido, mientras que los *fine-tuned* mejoran la adaptación de dominio, la integración del contexto y formato de salida.
 - Abiertos o propietarios: los abiertos ofrecen control y despliegue flexible, mientras que los propietarios simplifican el uso y el mantenimiento.
 - Tamaño del modelo: los grandes tienen un razonamiento mejor y un contexto más amplio. Los pequeños tienen un coste y latencia menor.

Capas de soporte

Además de los dos *pipelines*, un sistema en producción requiere orquestación, evaluación y monitorización continua, *cache* semántico para reducir coste y latencia, controles de seguridad para cumplimiento normativo y seguridad frente a inyecciones en el *prompt*, envenenamiento de dato o fugas de información. Todo ello se sostiene sobre una infraestructura de servicio y un *RAGOps stack* formado por capas críticas, esenciales y de mejora que aseguran operación, calidad y seguridad extremo a extremo [4].

1. Capas críticas: son las capas imprescindibles del *RAGOps stack*.

- Capa de datos: es la encargada de crear la base de conocimiento, para ello recolecta datos desde sistemas origen, los transforma y los almacena.
 - Capa de modelos: se incluyen los modelos de *embeddings*, LLMs y de las tareas. Incluye el entrenamiento y la optimización de inferencia.
 - Despliegue de modelos: pueden ser servicios gestionados, auto-hospedados o locales.
 - Orquestación de la aplicación: se encarga de coordinar la consulta, la recuperación de datos y la generación. Incluye soporte multiagente y automatización de flujos de trabajo.
2. Capas esenciales: son las capas que necesita el sistema, se encargan del rendimiento, la fiabilidad y de la seguridad.
- Capa de *prompt*: diseño, versión y evaluación de *prompts* para guiar al *LLM* y reducir las alucinaciones.
 - Capa de evaluación: se encarga de medir, de forma periódica, la relevancia de contexto, fidelidad y relevancia de la respuesta.
 - Capa de monitorización: observa el proceso del *RAG* para encontrar fallos. Para ello monitoriza la latencia y el error.
 - Seguridad y privacidad del *LLM*: es la encargada de emplear métodos para evitar el *prompt injection*. Algunos de los métodos que se emplean son validar las entradas, realizar encriptados y emplear control de accesos.
 - Capa de caché: emplea caché para reducir los costes y la latencia en consultas frecuentes.
3. Capas de mejora: son capas opcionales que pueden aportar beneficios dependiendo del caso de uso. Se centran en la eficiencia, usabilidad y escalabilidad del sistema.

3.6. Evaluación de la calidad del *RAG*

Se suele hacer mediante la tríada de evaluación propuesta por TruEra [7] (ver imagen de la triada 3.3). Estructura la calidad en tres comprobaciones entre consulta (*prompt*), contexto recuperado y respuesta. Complementariamente, se emplean métricas como relevancia, precisión y *recall*, y conjuntos *ground truth* para validación objetiva y monitorización continua en producción [4].

Las tres dimensiones de la tríada son:



Figura 3.3: Triada de evaluación TruEra.

- Relevancia del contexto: la información que se haya recuperado tiene que ver con la pregunta.
- *Groundedness* o fidelidad: la respuesta que da el *LLM* se basa de verdad en el contexto que le hemos dado, sin alucinaciones ni omisiones clave.
- Relevancia de la respuesta: la respuesta es útil y adecuada respecto a la intención y el contexto de la consulta original.

Aparte de las dimensiones de la tríada, otros aspectos relevantes que el sistema debe incorporar son:

- Robustez al ruido: el sistema debe distinguir aquellos documentos relacionados pero que sean poco útiles.
- Rechazo en negativo: el sistema no debe responder cuando no hay evidencias relevantes.
- Integración de información: el sistema debe ser capaz de sintetizar múltiples documentos que posean información relevante.
- Robustez contrafactual: el sistema tiene que detectar y evitar la información inexacta en el propio *knowledge base*

Hay dos herramientas principales de evaluación:

- *Retrieval-Augmented Generation Assessment* (RAGAs): es un *framework* que genera datos sintéticos y calcula métricas de la tríada. Es útil para ciclos rápidos [11].
- *Automated RAG Evaluation System*(ARES): es un *framework* con un funcionamiento similar al RAGAs, pero con mayor complejidad y cantidad de datos generados. Es más robusto [12].

4. Técnicas y herramientas

En este apartado se va a realizar una breve explicación de la metodología empleada, así como de las herramientas utilizadas para el desarrollo del proyecto. En dicha explicación se va a incluir una comparación con otras herramientas similares en la cual se muestre el motivo de la selección de dicha herramienta.

4.1. Metodologías

Scrum

La metodología *Scrum* es un marco de trabajo ágil orientado a la gestión y desarrollo de proyectos de manera iterativa e incremental. Se organiza en ciclos cortos denominados *sprints*, al final de los cuales se entrega un incremento funcional del producto (ver imagen 4.1 para ver el proceso *Scrum* completo). Estos *sprints* favorecen la mejora continua y la adaptación al cambio, además, aseguran una entrega temprana de valor. A diferencia de metodologías tradicionales, *Scrum* aporta mayor flexibilidad y retroalimentación constante [13].

4.2. Repositorio

Para alojar el repositorio se han valorado distintas alternativas como [Bitbucket](#), [GitHub](#), [GitLab](#) o [Trello](#). Finalmente se ha elegido GitHub que es una plataforma basada en la nube que permite alojar repositorios de código. GitHub ofrece funciones muy útiles como ramas (*branches*), *pull request*, seguimiento de cambios (*commits*), historial del proyecto, gestión

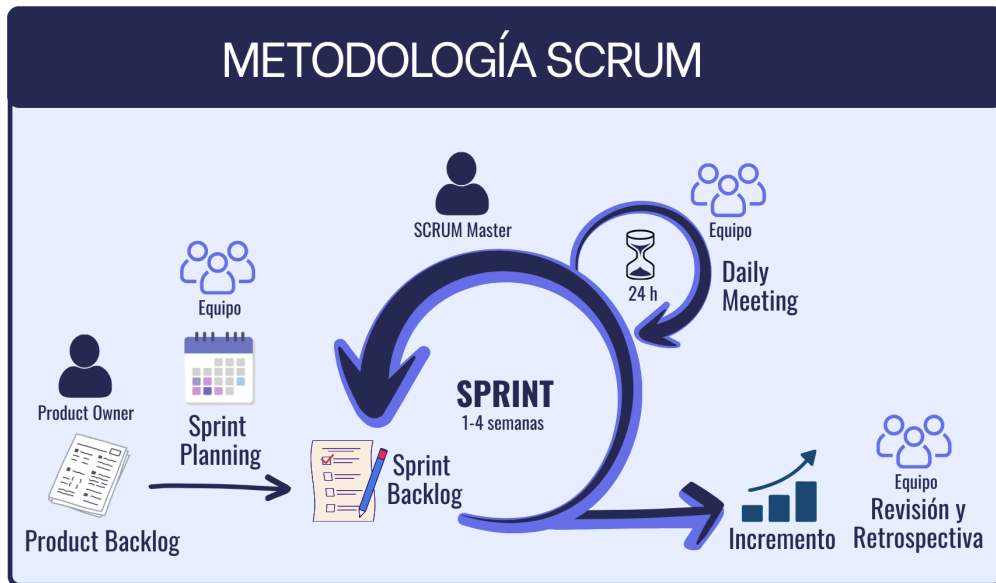


Figura 4.1: Metodología *Scrum*.

de incidencias (*issues*). Además, permite la colaboración entre usuarios, lo que es muy útil para la revisión del proyecto.

Otra ventaja es que GitHub se integra con herramientas de integración y despliegue continuo, lo que facilita la automatización de pruebas y la entrega de *software*. Asimismo, permite incluir documentación directamente en el repositorio, ya sea mediante archivos *README* o a través de *Wikis*, lo que mejora la claridad y la reproducibilidad del trabajo. Finalmente, al tratarse de la plataforma más utilizada a nivel mundial, cuenta con abundante documentación, lo que la convierte en una alternativa sólida y con amplio soporte [2].

4.3. Gestión de tareas

Para la gestión de las tareas del proyecto se han valorado diversas herramientas, como **ZenHub**, **GitHub Projects** o **Asana**. Al final se ha optado por usar ZenHub que es una herramienta de gestión de proyectos fácilmente integrable con GitHub. Se ha elegido porque tiene mayor funcionalidad que GitHub Projects. Ambos permiten hacer tableros Kanban para organizar las tareas (*to-do*, *in progress*, *doing*). Pero, ZenHub, además, permite asignar *story points* a las tareas y generar los gráficos *burndown* de los *sprints*.

4.4. L^AT_EX

Para la edición de los documentos en L^AT_EX se ha considerado usar [T_EXMaker](#), [T_EXStudio](#), [Visual Studio Code](#) y [Overleaf](#). Finalmente se ha optado por usar T_EXMaker, ya que es un editor multiplataforma, gratuito y de código abierto. Tiene un visor PDF integrado, así como herramientas para gestionar la bibliografía mediante BibT_EX. Además, tiene una interfaz simple que facilita el aprendizaje, incorpora funciones como el autocompletado, el plegado de código y la detección de errores de compilación, sin necesidad de instalar complementos adicionales.

Como distribución L^AT_EX para procesar los documentos .tex se ha valorado el uso de [MikT_EX](#) y [T_EXLive](#), al final se ha elegido usar MikT_EX ya que permite realizar una instalación mínima e ir añadiendo automáticamente los paquetes necesarios durante la compilación. Además, presenta un buen soporte para Windows, lo que se adapta bien a mi entorno de trabajo [15].

4.5. Web Scraping

Para el *web scraping* se ha valorado usar [Selenium](#) y [Playwright](#). Selenium controla navegadores reales y es un estándar veterano, pero Playwright es más completo. Ya que ofrece automatización multi-navegador con contextos aislados, funcionamiento headless, espera automática de elementos y APIs consistentes, lo que lo vuelve más robusto y rápido para páginas con JavaScript pesado y flujos de login. En la tabla 4.1 se muestran las ventajas y desventajas de cada uno.

Aunque *Selenium* sigue siendo una apuesta segura, para *scraping* moderno con multiples sesiones y depuración rápida, *Playwright* requiere menos código y es más robusto.

4.6. Gestión de dependencias y entornos Python

El proyecto requiere entornos aislados y reproducibles que faciliten el trabajo local, la integración continua y la generación de entregables. Se ha valorado utilizar [Poetry](#) y [uv](#). Poetry es una herramienta madura de gestión de dependencias y empaquetado que centraliza la configuración en *pyproject.toml*. Crea y gestiona entornos virtuales de forma automática, mantiene un archivo de bloqueo (*poetry.lock*) para asegurar instalaciones

Ventajas	Desventajas
<i>Selenium</i> [10]	
■ Utiliza el estándar W3C <i>WebDriver</i> [9], lo que aporta interoperabilidad y soporte a largo plazo. ■ Multilenguaje y ecosistema maduro. Permite una integración fácil. ■ Es robusto para escalado distribuido.	■ Es propenso a fallos (<i>flakiness</i>) si no se dominan las esperas. ■ La intercepción de red en menos homogénea entre navegadores.
<i>Playwright</i> [14]	
■ Es muy estable y rápido en <i>webs</i> con <i>JavaScript</i> pesado. ■ Presenta multitud de herramientas integradas como <i>Trace Viewer</i>, capturas de pantalla y videos. ■ Contiene selectores potentes y permite manejar los <i>inframes</i> de forma sencilla. ■ Tiene un control de contexto aislado muy cómodo. ■ Su modelos asíncrono permite realizar acciones concurrentes sin bloquear el hilo empleando la <i>API async/await</i>.	■ No utiliza el estándar <i>WebDriver</i>.

Tabla 4.1: Ventajas y desventajas de *Selenium* y *Playwright*.

deterministas, permite definir grupos de dependencias, así como, generar de manera automática el *requirements.txt*. Por su parte *uv* es un gestor rápido que, al igual que *Poetry*, tiene soporte para *pyproject.toml*. Posee un buen rendimiento y una elevada velocidad gracias al uso de *caches* eficientes. Finalmente, se ha optado por usar *Poetry* por su cobertura integral del ciclo de vida dentro de un flujo coherente y estable.

4.7. Base de datos vectorial

Una parte fundamental del desarrollo del *RAG* es la implementación de la base de datos vectorial. Se han valorado emplear diversas bases de datos, pero las dos que más se han estudiado son *Qdrant* y *pgvector*. *Qdrant* es una base de datos vectorial especializada, diseñada desde cero para búsquedas de similitud de alta concurrencia y baja latencia. Presenta equilibrio entre el rendimiento, la latencia y el tiempo de indexado [3]. Por su parte, *pgvector* es una extensión de *PostgreSQL* que permite almacenar embeddings en columnas vectoriales y realizar búsquedas de similitud sobre una base de datos relacional ya existente. Es útil cuando la aplicación ya se apoya en este gestor, ya que simplifica la infraestructura [16]. En la tabla 4.2 se muestran las principales ventajas y desventajas de cada opción.

	Ventajas	Desventajas
<i>Qdrant</i>	■ Diseñada específicamente como base de datos vectorial, con índices optimizados para búsquedas de similitud y buen compromiso entre RPS, latencia y tiempo de indexado [3]. ■ Soporta filtrado avanzado por metadatos y búsquedas híbridas (vectoriales más filtros), lo que facilita consultas complejas en <i>RAG</i>. ■ Ofrece despliegue sencillo mediante contenedores y opciones gestionadas, con APIs HTTP y gRPC bien documentadas. ■ Integración directa con librerías y orquestadores de <i>RAG</i>, lo que reduce el código necesario para conectarla con el resto del sistema.	■ Introduce un nuevo servicio en la arquitectura: es necesario administrar una base de datos adicional a la relacional. ■ Requiere aprender herramientas y mecanismos de operación específicos (copias de seguridad, monitorización, actualización de índices, etc.). ■ Para casos de uso pequeños puede resultar más compleja que reutilizar la base de datos transaccional existente.

Ventajas	Desventajas
<i>pgvector</i>	
■ Permite reutilizar una instancia de PostgreSQL ya desplegada, unificando datos transaccionales y embeddings en el mismo sistema. ■ Aprovecha todo el ecosistema SQL: transacciones, <i>joins</i>, vistas, roles de usuario y mecanismos estándar de replicación y copia de seguridad. ■ Simplifica la operación en entornos reducidos o con restricciones de infraestructura, al requerir un único motor de base de datos.	■ El rendimiento en búsquedas vectoriales a gran escala (millones de vectores y alta concurrencia) suele ser inferior al de motores especializados. ■ Dispone de menos funcionalidades específicas de bases de datos vectoriales (gestión de colecciones, métricas especializadas o afinado fino de índices). ■ La configuración óptima de índices y parámetros depende de la versión de PostgreSQL y de la extensión, y puede requerir un ajuste cuidadoso.

Tabla 4.2: Ventajas y desventajas de *Qdrant* y *pgvector*.

Para realizar el proyecto se ha optado por utilizar *Qdrant*. Aunque *pgvector* presenta características interesantes cuando se desea mantener toda la información en una única base de datos relacional, *Qdrant* presenta un diseño especializado. Así como un mejor equilibrio entre rendimiento y latencia. Además, *Qdrant* se utiliza en proyectos reales lo que lo convierte en una muy buena opción para implementar el sistema *RAG*.

4.8. Postgres

4.9. Docker

5. Aspectos relevantes del desarrollo del proyecto

Este apartado pretende recoger los aspectos más interesantes del desarrollo del proyecto, comentados por los autores del mismo. Debe incluir desde la exposición del ciclo de vida utilizado, hasta los detalles de mayor relevancia de las fases de análisis, diseño e implementación. Se busca que no sea una mera operación de copiar y pegar diagramas y extractos del código fuente, sino que realmente se justifiquen los caminos de solución que se han tomado, especialmente aquellos que no sean triviales. Puede ser el lugar más adecuado para documentar los aspectos más interesantes del diseño y de la implementación, con un mayor hincapié en aspectos tales como el tipo de arquitectura elegido, los índices de las tablas de la base de datos, normalización y desnormalización, distribución en ficheros³, reglas de negocio dentro de las bases de datos (EDVHV GH GDWRV DFWLYDV), aspectos de desarrollo relacionados con el WWW... Este apartado, debe convertirse en el resumen de la experiencia práctica del proyecto, y por sí mismo justifica que la memoria se convierta en un documento útil, fuente de referencia para los autores, los tutores y futuros alumnos.

6. Trabajos relacionados

En este apartado se exponen los trabajos relacionados con el TFG que han sido publicados hasta la fecha.

6.1. TFG relacionados

Retrieval-Augmented Generation para la Extracción de Información de Documentos Inteligente

En este TFG se desarrolla un sistema RAG orientado al ámbito sanitario con el propósito de optimizar la recuperación y generación de información médica. El trabajo plantea una arquitectura basada en la recuperación y generación, que indexa documentación médica, crea *embeddings* y orquesta *prompts* para aportar contexto fiable al modelo generativo. Emplea técnicas de *chunking* con solapamiento y un enfoque *MultiQuery Retriever* para mejorar la cobertura y la precisión de la recuperación previa a la síntesis con un LLM. La solución se implementa sobre la API de *Azure OpenAI*, utilizando *LangChain* y *FAISS* como motor de búsqueda vectorial, e integra una interfaz web en *Flask* para simular consultas clínicas reales. Para la evaluación del sistema se emplea el *framework RAGAS*, valorando métricas de precisión, relevancia y coherencia de las respuestas generadas, demostrando mejoras significativas frente a enfoques puramente generativos [6].

NotebookLM

7. Conclusiones y Líneas de trabajo futuras

Todo proyecto debe incluir las conclusiones que se derivan de su desarrollo. Éstas pueden ser de diferente índole, dependiendo de la tipología del proyecto, pero normalmente van a estar presentes un conjunto de conclusiones relacionadas con los resultados del proyecto y un conjunto de conclusiones técnicas. Además, resulta muy útil realizar un informe crítico indicando cómo se puede mejorar el proyecto, o cómo se puede continuar trabajando en la línea del proyecto realizado.

Bibliografía

- [1] Alejandro Alija. Técnicas *RAG*: cómo funcionan y ejemplos de casos de uso. <https://datos.gob.es/es/blog/tecnicas-rag-como-funcionan-y-ejemplos-de-casos-de-uso>, 2024. [Online; Accedido 6-Febrero-2026].
- [2] GitHub Inc. Acerca de github y git. <https://docs.github.com/es/get-started/start-your-journey/about-github-and-git>, 2025. [Online; Accedido 22-Septiembre-2025].
- [3] Paul Iusztin and Maxime Labonne. *The LLM Engineer's Handbook*. Packt Publishing, October 2024. ISBN 9781836200079. [Consultado 30-Octubre-2025 a 27-Noviembre-2025].
- [4] Abhinav Kimothi. *A Simple Guide to Retrieval Augmented Generation*. Manning Publications, July 2025. ISBN 9781633435858. [Consultado 8-Septiembre-2025 a 26-Septiembre-2025].
- [5] Martijn Koster, G. Illyes, H. Zeller, and L. Sassman. Robots exclusion protocol (rfc 9309). <https://datatracker.ietf.org/doc/html/rfc9309>, 2022. [Online; Accedido 07-Octubre-2025].
- [6] Daniel Laparra Osca. Retrieval-augmented generation para la extracción de información de documentos inteligente. Trabajo fin de grado, Universitat Politècnica de València, Escuela Técnica Superior de Ingeniería de Telecomunicación, 2024. URL <https://riunet.upv.es/server/api/core/bitstreams/c775be7d-8724-476b-b0a4-73e055d03d50/content>. [Accedido 02-Octubre-2025].

- [7] Lofred Madzou. What is the rag triad? <https://truera.com/ai-quality-education/generative-ai-rags/what-is-the-rag-triad/>. [Online; Accedido 24-Septiembre-2025].
- [8] Columbia University Mailman School of Public Health. Web scraping. <https://www.publichealth.columbia.edu/research/population-health-methods/web-scraping>, -. [Online; Accedido 28-Septiembre-2025].
- [9] W3C Working Group on Browser Testing and Tools. Webdriver — w3c working draft (level 2). <https://www.w3.org/TR/webdriver2/>, 2025. [Online; Accedido 15-October-2025].
- [10] Selenium Project. Selenium documentation. <https://www.selenium.dev/documentation/>, 2025. [Online; Accedido 10-October-2025].
- [11] Ragas. Ragas documentation. <https://docs.ragas.io/en/stable/>, 2025. [Online; Accedido 05-October-2025].
- [12] Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. Ares: An automated evaluation framework for retrieval-augmented generation systems. <https://github.com/stanford-futuredata/ARES>, 2023. [Online, Accedido 05-October-2025].
- [13] Ken Schwaber and Jeff Sutherland. The scrum guide. <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>, 2020. [Online; Accedido 22-Septiembre-2025].
- [14] Microsoft Playwright Team. Playwright documentation: Actionability. <https://playwright.dev/docs/actionability>, 2025. [Online; Accedido 10-October-2025].
- [15] Joseph Wright. Tex on windows: Miktex or tex live? *TUGboat*, 33(1):53, 2012. URL <https://www.tug.org/TUGboat/tb33-1/tb103wright.pdf>. [Online; Accedido 22-Septiembre-2025].
- [16] Zilliz. Qdrant vs. pgvector. <https://zilliz.com/comparison/qdrant-vs-pgvector>, 2025. [Online; Accedido 10-Noviembre-2025].